



36-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Models va ORM texnologiyasi. Model yaratish va migratsiyalar

Model — jadvalning Python klassi, maydon turlari, ForeignKey bog'lanishlari, __str__ va Meta; makemigrations va migrate; ORM bilan CRUD: create, all, filter, get, order_by, update, delete; get_object_or_404; view'larni bazaga ulash.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **Model** — ma'lumotlar bazasi jadvalini Python klassi sifatida yozishni;
- Maydon turlari va jadvallar o'rtasidagi bog'lanishlarni (`ForeignKey`);
- **Migratsiya** — modelni haqiqiy jadvalga aylantirish jarayonini;
- **ORM** yordamida SQL yozmasdan ma'lumot qo'shish, o'qish, filtrlash, o'zgartirish va o'chirishni;
- Blog sahifalarini haqiqiy PostgreSQL bazasiga ulashni.

ORM nima?

III bobda SQL yozdik, 32-mavzuda esa uni Python ichida satr sifatida yubordik. Django'da yanada qulay yo'l bor — **ORM** (Object-Relational Mapping): jadvallar — **klasslar**, qatorlar — **obyektlar**, SQL o'rniga — **Python kodi**.

SQL	Django ORM
<pre>SELECT * FROM blog_maqola;</pre>	<pre>Maqola.objects.all()</pre>
<pre>SELECT * FROM blog_maqola WHERE id = 5;</pre>	<pre>Maqola.objects.get(id=5)</pre>

```
... WHERE sarlavha  
ILIKE '%futbol%'  
ORDER BY yaratilgan  
DESC
```

```
Maqola.objects.filter(sarlavha__icontains="futbol").order_by("-  
yaratilgan")
```

```
INSERT INTO  
blog_maqola (...)  
VALUES (...);
```

```
Maqola.objects.create(sarlavha="...", ...)
```



O'xshatish: tarjimon

Siz o'zbekcha (Python) gapirasiz, baza esa faqat inglizcha (SQL) tushunadi. ORM — tarjimon: sizning `Maqola.objects.filter(...)` gapingizni to'g'ri va xavfsiz SQL'ga o'giradi. Bonus: SQL injection'dan avtomatik himoya (32-mavzu) va bazani almashtirsangiz (SQLite → PostgreSQL) kod o'zgarmaydi.

Model yaratish

blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
from django.urls import reverse


class Kategoriya(models.Model):
    nomi = models.CharField("Nomi", max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    class Meta:
        verbose_name = "Kategoriya"
        verbose_name_plural = "Kategoriyalar"
        ordering = ["nomi"]

    def __str__(self):
        return self.nomi


class Maqola(models.Model):
    sarlavha = models.CharField("Sarlavha", max_length=200)
    matn = models.TextField("Matn")
    kategoriya = models.ForeignKey(
        Kategoriya,
        on_delete=models.PROTECT,          # kategoriyada maqola bo'lsa –
o'chirib bo'lmaydi
        related_name="maqolalar",
    )
    muallif = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="maqolalar")
    chop_etilgan = models.BooleanField("Chop etilgan", default=True)
    korishlar = models.PositiveIntegerField(default=0)
    yaratilgan = models.DateTimeField(auto_now_add=True)    # yaratilganda
avtomatik
    yangilangan = models.DateTimeField(auto_now=True)      # har saqlanganda
avtomatik

    class Meta:
        verbose_name = "Maqola"
        verbose_name_plural = "Maqolalar"
```

```

        ordering = ["-yaratilgan"] # yangilari
    birinchi

    def __str__(self):
        return self.sarlavha

    def get_absolute_url(self):
        return reverse("blog:maqola", args=[self.id])

```

Har bir model — `models.Model` dan **meros** olgan klass (25-mavzu!). Har bir atribut — jadval **ustuni**. `id` ustunini yozmaymiz — Django uni avtomatik qo'shadi.

Asosiy maydon turlari

Django maydoni	PostgreSQL turi	Qachon
<code>CharField(max_length=...)</code>	<code>VARCHAR(n)</code>	Qisqa matn: ism, sarlavha
<code>TextField()</code>	<code>TEXT</code>	Uzun matn: maqola
<code>IntegerField()</code> , <code>PositiveIntegerField()</code>	<code>INTEGER</code>	Butun son
<code>DecimalField(max_digits, decimal_places)</code>	<code>NUMERIC</code>	Pul
<code>BooleanField()</code>	<code>BOOLEAN</code>	Ha/yo'q
<code>DateTimeField()</code> , <code>DateField()</code>	<code>DATE</code> , <code>TIMESTAMP</code>	Sana, vaqt
<code>EmailField()</code> , <code>URLField()</code> , <code>SlugField()</code>	<code>VARCHAR</code> + tekshiruv	Email, havola, slug
<code>ImageField(upload_to=...)</code>	<code>VARCHAR</code> (fayl yo'li)	Rasm (<code>pip install Pillow</code> kerak)
<code>ForeignKey(Model, on_delete=...)</code>	FK ustun <code>..._id</code>	Birga-ko'p bog'lanish
<code>ManyToManyField(Model)</code>	Oraliq jadval (avtomatik)	Ko'pga-ko'p (teglar)

Maydon parametrlari

- `null=True` — bazada NULL bo'lishi mumkin; `blank=True` — formada bo'sh qoldirish mumkin.
- `default=...` — standart qiymat; `unique=True` — takrorlanmas; `choices=[...]` — tanlov ro'yxati.
- Birinchi argument (`"Sarlavha"`) — admin panel va formalarda ko'rinadigan chiroyli nom.

on_delete — 29-mavzudagi yaxlitlik qoidalari

Django	SQL ma'nosi	Bizning misolda
<code>CASCADE</code>	<code>ON DELETE CASCADE</code>	Foydalanuvchi o'chsa — maqolalari ham
<code>PROTECT</code>	<code>RESTRICT</code>	Maqolasi bor kategoriyanı o'chirib bo'lmaydi
<code>SET_NULL</code>	<code>SET NULL</code> (<code>null=True</code> kerak)	—

Migratsiyalar — modeldan jadvalga

Model — faqat Python kodi. Bazada jadval paydo bo'lishi uchun ikki qadam kerak:

Terminal

```
python manage.py makemigrations # 1) model o'zgarishlaridan "reja"
(migratsiya fayli) yaratish
python manage.py migrate # 2) rejani bazaga qo'llash (CREATE TABLE
...)
```

Natija

```
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Kategoriya
    - Create model Maqola
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0001_initial... OK
```



O'xshatish: qurilish chizmasi

`models.py` — uyning yangi loyihasi. `makemigrations` — quruvchilar uchun "nimani o'zgartirish kerak" degan ish rejasini yozish ("2-qavatga xona qo'shilsin"). `migrate` — rejani haqiqatda bajarish. Migratsiya fayllari — barcha o'zgarishlar tarixi (Git kabi!).

pgAdmin'da `blog_kategoriya` va `blog_maqola` jadvallarini toping (nom: `ilova_model`). `blog_maqola` da `kategoriya_id` va `muallif_id` FK ustunlariga e'tibor bering. Django qanday SQL yozganini ko'rish: `python manage.py sqlmigrate blog 0001`.



Qoida

`models.py` ni har o'zgartirganingizda (maydon qo'shish, nomini o'zgartirish) — **`makemigrations`** va **`migrate`**. Migratsiya fayllarini qo'lda tahrirlamang va Git'ga commit qiling.

ORM bilan ishlash: Django shell

Terminal

```
python manage.py createsuperuser      # admin foydalanuvchi (maqola muallifi uchun ham kerak)
python manage.py shell                 # loyiha ichidagi Python konsoli
```

Create — yaratish

Python

```
from blog.models import Kategoriya, Maqola
from django.contrib.auth.models import User

admin = User.objects.first()

sport = Kategoriya.objects.create(nomi="Sport", slug="sport")
tadbir = Kategoriya.objects.create(nomi="Tadbirlar", slug="tadbirlar")

m = Maqola.objects.create(
    sarlavha="Futbol bo'yicha g'alaba",
    matn="Maktabimiz jamoasi shahar chempionatida birinchi o'rinni egalladi!",
    kategoriya=sport,
    muallif=admin,
)
print(m.id, m.yaratilgan)

# Ikkinchi usul: obyekt yaratib, keyin saqlash
m2 = Maqola(sarlavha="Bilimlar bellashuvi", matn="...", kategoriya=tadbir,
muallif=admin)
m2.save()
```

Read — o'qish

Python

```
Maqola.objects.all() # barchasi (QuerySet)
Maqola.objects.count() # soni
Maqola.objects.get(id=1) # aynan bittasi (topilmasa – DoesNotExist xatosi!)
Maqola.objects.first(), Maqola.objects.last()

Maqola.objects.filter(chop_etilgan=True) # WHERE chop_etilgan = TRUE
Maqola.objects.filter(kategoriya__slug="sport") # bog'langan jadval bo'yicha (JOIN!)
Maqola.objects.filter(sarlavha__icontains="futbol") # ILIKE '%futbol%'
Maqola.objects.filter(korishlar__gte=100) # ≥ 100
Maqola.objects.filter(yaratilgan__year=2026)
Maqola.objects.exclude(chop_etilgan=False) # NOT
Maqola.objects.order_by("-korishlar")[:5] # ORDER BY ... DESC LIMIT 5

m = Maqola.objects.get(id=1)
print(m.sarlavha, m.kategoriya.nomi, m.muallif.username) # FK orqali bog'langan obyektga o'tish
sport.maqolalar.all() # teskari yo'nalish (related_name)
```

Lookup	SQL
maydon=x	= x
maydon__gt / __gte / __lt / __lte	> / ≥ / < / ≤
maydon__contains / __icontains	LIKE '%x%' / ILIKE
maydon__startswith	LIKE 'x%'
maydon__in=[1, 2]	IN (1, 2)
maydon__isnull=True	IS NULL
fk__maydon	JOIN orqali bog'langan jadval ustuni

Update va Delete

Python

```
m = Maqola.objects.get(id=1)
m.sarlavha = "Futbol: shahar chempionlari!"
m.korishlar += 1
m.save() # UPDATE

Maqola.objects.filter(kategoriya__slug="sport").update(chop_etilgan=False) #
ko'pini birdan

Maqola.objects.get(id=2).delete() # DELETE
```

Agregatsiya (30-mavzu Django'da)

Python

```
from django.db.models import Count, Sum, Avg

Maqola.objects.aggregate(jami=Sum("korishlar"), ortacha=Avg("korishlar"))
# {'jami': 340, 'ortacha': 85.0}

Kategoriya.objects.annotate(soni=Count("maqolalar")) # GROUP BY: har bir
kategoriyadagi maqolalar soni
for k in Kategoriya.objects.annotate(soni=Count("maqolalar")):
    print(k.nomi, k.soni)
```



QuerySet "dangasa": `Maqola.objects.filter(...)` yozilganda baza hali so'ralmaydi — faqat natija haqiqatan kerak bo'lganda (sikl, `print`, shablon) bitta SQL yuboriladi. Shu tufayli `filter().exclude().order_by()` zanjiri samarali ishlaydi.

View'larni bazaga ulash

Endi 34–35-mavzulardagi **MAQOLALAR** ro'yxatini o'chirib, haqiqiy modeldan foydalanamiz:

```

from django.shortcuts import render, get_object_or_404
from django.db.models import Q, F
from .models import Maqola, Kategoriya

def bosh_sahifa(request):
    maqolalar =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    return render(request, "blog/bosh_sahifa.html", {"maqolalar": maqolalar})

def maqola(request, id):
    m = get_object_or_404(Maqola, id=id, chop_etilgan=True)      # topilmasa –
    avtomatik 404
    Maqola.objects.filter(id=m.id).update(korishlar=F("korishlar") + 1)  #
    ko'rishlar +1
    return render(request, "blog/maqola.html", {"maqola": m})

def kategoriya(request, nomi):
    kat = get_object_or_404(Kategoriya, slug=nomi)
    return render(request, "blog/kategoriya.html", {
        "kategoriya": kat,
        "maqolalar": kat.maqolalar.filter(chop_etilgan=True),
    })

def qidiruv(request):
    q = request.GET.get("q", "").strip()
    natija = Maqola.objects.none()
    if q:
        natija = Maqola.objects.filter(
            Q(sarlavha__icontains=q) | Q(matn__icontains=q),      # Q – YOKI
            chop_etilgan=True,
        )
    return render(request, "blog/qidiruv.html", {"q": q, "maqolalar": natija})

```

Shablonlar deyarli o'zgarmaydi — dict kalitlari o'rniga model atributlari: `{{ m.sarlavha }}`, `{{ m.kategoriya.nomi }}`, `{{ m.yaratilgan|date:"d.m.Y" }}`, havola uchun `{{ m.get_absolute_url }}`.

```
{# m — Maqola obyekti #}
<article class="karta">
  <a class="belgi" href="{% url 'blog:kategoriya' m.kategoriya.slug %}">{{
m.kategoriya.nomi }}</a>
  <h3><a href="{{ m.get_absolute_url }}">{{ m.sarlavha }}</a></h3>
  <p>{{ m.matn|truncatewords:20 }}</p>
  <small>👉 {{ m.muallif.username }} · 📅 {{ m.yaratilgan|date:"d.m.Y" }} · 👁
{{ m.korishlar }}</small>
</article>
```



`select_related("kategoriya", "muallif")` — kategoriya va muallifni **bitta JOIN so'rov** bilan oladi. Usiz har bir karta uchun alohida so'rov ketardi (10 ta maqola = 21 ta so'rov). Bu "N+1 muammosi" deyiladi.

Amaliy mashg'ulot

- 1 `Kategoriya` va `Maqola` modellarini yozing, `makemigrations` + `migrate`. pgAdmin'da jadvallarni ko'ring.
- 2 `createsuperuser` bilan admin yarating.
- 3 `shell` orqali 3 ta kategoriya va 6 ta maqola yarating. ORM so'rovlarini mashq qiling: sport maqolalari, sarlavhasida "maktab" bor maqolalar, eng ko'p ko'rilgan 3 tasi.
- 4 View'larni bazaga ulang, `MAQOLALAR` ro'yxatini o'chiring. `kategoriya.html` va `qidiruv.html` shablonlarini (`base.html` dan meros, kartalar uchun `_karta.html`) yarating. `base.html` dagi menyuga qidiruv formasini qo'shing: `<form action="{% url 'blog:qidiruv' %}"><input name="q"></form>`. Sayt endi bazadagi ma'lumotni ko'rsatishini tekshiring.
- 5 Bosh sahifaga kategoriyalar ro'yxatini maqolalar soni bilan qo'shing (`annotate(Count(...))`).
- 6 **Model o'zgartirish:** `Maqola` ga `rasm = models.ImageField(upload_to="maqolalar/", blank=True)` qo'shing (`pip install Pillow`), migratsiya qiling. `settings.py` ga `MEDIA_URL = "media/"`, `MEDIA_ROOT = BASE_DIR / "media"`; `config/urls.py` oxiriga:

Python

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT) # faqat DEBUG rejimida
```

Shablonda: `{% if m.rasm %}{% endif %}`.

Ko'p uchraydigan xatolar

- ✗ `no such table / relation "blog_maqola" does not exist` — `migrate` bajarilmagan.
- ✗ `No changes detected` — ilova `INSTALLED_APPS` da yo'q yoki fayl saqlanmagan.
- ✗ `Maqola matching query does not exist` — `get()` hech narsa topmadi; view'larda `get_object_or_404` ishlatilg.
- ✗ `get() returned more than one` — `get` faqat bitta natija uchun; ko'p bo'lsa `filter`.
- ✗ Yangi majburiy maydon qo'shganda migratsiya standart qiymat so'raydi — `default=...` yoki `null=True` bering.

O'zingizni tekshiring

? ORM nima?

- A. Ma'lumotlar bazasi turi
- B. Jadvallar bilan Python klass va obyektlari orqali ishlash texnologiyasi
- C. HTML shablon tili
- D. Server dasturi

? Modelni o'zgartirgandan keyin bajariladigan ikki buyruq?

- A. `runserver` va `shell`
- B. `startapp` va `migrate`
- C. `makemigrations` va `migrate`
- D. `createsuperuser` va `collectstatic`

? Uzun maqola matni uchun eng mos maydon?

- A. `TextField`
- B. `CharField(max_length=50)`
- C. `SlugField`
- D. `IntegerField`

? Sarlavhasida "olimpiada" so'zi bor maqolalar (katta-kichik harfga qaramay)?

- A. `Maqola.objects.get(sarlavha="olimpiada")`
- B. `Maqola.objects.all("olimpiada")`
- C. `Maqola.objects.filter(sarlavha="%olimpiada%")`
- D. `Maqola.objects.filter(sarlavha__icontains="olimpiada")`

? Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya?

- A. `Maqola.objects.get()`
- B. `get_object_or_404()`
- C. `find_or_404()`
- D. `Http404()`

? "Har bir maqolaning bitta kategoriyasi, kategoriyada ko'p maqola" — qaysi maydon?

- A. `ManyToManyField`
- B. `OneToOneField`
- C. `ForeignKey` (Maqola modelida)
- D. `CharField`

Uyga vazifa

1. **Kutubxona tizimi (22-loyiha) — modellar:** `Muallif`, `Kitob` (FK muallif, janr — `choices`, yil, sahifalar, mavjud), `Oquvchi`. Migratsiya va shell orqali ma'lumotlar.
2. **Teglar:** `Teg` modeli va `Maqola.teglar = models.ManyToManyField(Teg, blank=True)`.
Shell'da: `m.teglar.add(t1, t2)`, `Maqola.objects.filter(teglar__nomi="olimpiada")`.

Yodda tuting

- Model — `class X(models.Model)`; maydonlar — ustunlar; `__str__`, `Meta`, `get_absolute_url`.
- `ForeignKey(..., on_delete=CASCADE/PROTECT/SET_NULL, related_name=...)`, `ManyToManyField`.
- `makemigrations` → `migrate` — har model o'zgarishidan keyin.
- ORM: `create`, `all`, `get`, `filter(maydon__lookup=...)`, `exclude`, `order_by`, `save`, `update`, `delete`, `annotate/aggregate`.
- View'larda `get_object_or_404`, `select_related`.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. ORM nima? — **B) Jadvallar bilan Python klass va obyektlari orqali ishlash texnologiyasi**
2. Modelni o'zgartirgandan keyin bajariladigan ikki buyruq? — **C) makemigrations va migrate**
3. Uzun maqola matni uchun eng mos maydon? — **A) TextField**
4. Sarlavhasida "olimpiada" so'zi bor maqolalar (katta-kichik harfga qaramay)? — **D) `Maqola.objects.filter(sarlavha__icontains="olimpiada")`**
5. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya? — **B) `get_object_or_404()`**
6. "Har bir maqolaning bitta kategoriyasi, kategoriyada ko'p maqola" — qaysi maydon? — **C) ForeignKey (Maqola modelida)**